

UNIVERSIDADE ESTADUAL DE FEIRA DE SANTANA
Departamento de Tecnologia
Curso de Engenharia de Computação
Disciplina: TEC499 — Sistemas Digitais | Professor: Angelo Duarte

NÚCLEO DE UM COPROCESSADOR GRÁFICO EM FPGA

para um Console Digital Dedicado na Plataforma DE1-SoC

Bruna de Almeida Nascimento
Carlos Daniel da Silva Jesus
Diego Mercês Almeida

Feira de Santana — Bahia
2026

1. Introdução

Este relatório apresenta a versão atualizada do projeto de um coprocessador gráfico dedicado, implementado em FPGA na placa Terasic DE1-SoC (Intel/Altera Cyclone V), desenvolvido como Problema Baseado em Aprendizagem (PBL) da disciplina TEC499 — Sistemas Digitais. O núcleo é descrito inteiramente em Verilog e gera, de forma autônoma, um sinal de vídeo VGA de 640x480 pixels a 60 Hz a partir de uma resolução lógica interna de 320x240, compondo três camadas gráficas independentes — background (tilemap), sprites e polígonos rasterizados — cuja seleção e edição são controladas diretamente pelas chaves (SW) e botões (KEY) físicos da placa.

Esta versão do documento substitui a anterior e foi elaborada a partir de uma reavaliação completa do código-fonte atual do projeto, tomado aqui como fonte de verdade sobre a arquitetura, os módulos, as memórias, a pinagem e as limitações reais do sistema. O material de apresentação (slides) e o relatório anterior foram usados apenas como referência de estrutura, organização e nível de detalhamento; todo o conteúdo técnico foi conferido diretamente nos arquivos `.v`, `.mif`, `.qsf` e `.qip` do projeto. Nos pontos em que a descrição anterior divergia do código, prevaleceu o código; essas divergências são apontadas explicitamente ao longo do texto.

2. Fundamentação e Conceitos de Arquitetura

A arquitetura do núcleo segue três princípios de projeto de motores gráficos em hardware, mantidos da fundamentação original e confirmados na implementação atual:

- **Datapath vs. Controle:** um datapath de vídeo (contadores de varredura, cálculo de endereços de memória e comparação de coordenadas) opera continuamente, pixel a pixel, enquanto a lógica de controle (a FSM de modos) responde apenas a eventos das chaves e botões. Essa separação mantém a geração de vídeo estável independentemente das ações do usuário.
- **ROM vs. RAM:** dados visuais fixos — os padrões gráficos dos tiles e dos sprites — são gravados em memórias ROM geradas a partir de arquivos `.mif` e sintetizadas como blocos M10K (megafunção ``altsyncram`, IP "ROM: 1-PORT"). Dados que mudam durante a demonstração (posição e atributos dos sprites, posição de rolagem do cenário, vértices dos polígonos) ficam em registradores internos dos próprios módulos, atualizados a cada ciclo de clock.
- **Sincronismo de vídeo:** a base de tempo de 25 MHz usada por todo o núcleo é obtida dividindo por dois o clock de 50 MHz da placa. Como a resolução lógica interna (320x240) é metade da resolução física VGA (640x480), cada pixel lógico é exibido como um bloco de 2x2 pixels físicos — a divisão de coordenada é feita por deslocamento de 1 bit, sem custo de hardware adicional.

3. Arquitetura Geral do Sistema

O diagrama a seguir representa a arquitetura efetivamente implementada em `main.v`, com os módulos nomeados exatamente como aparecem no código-fonte:

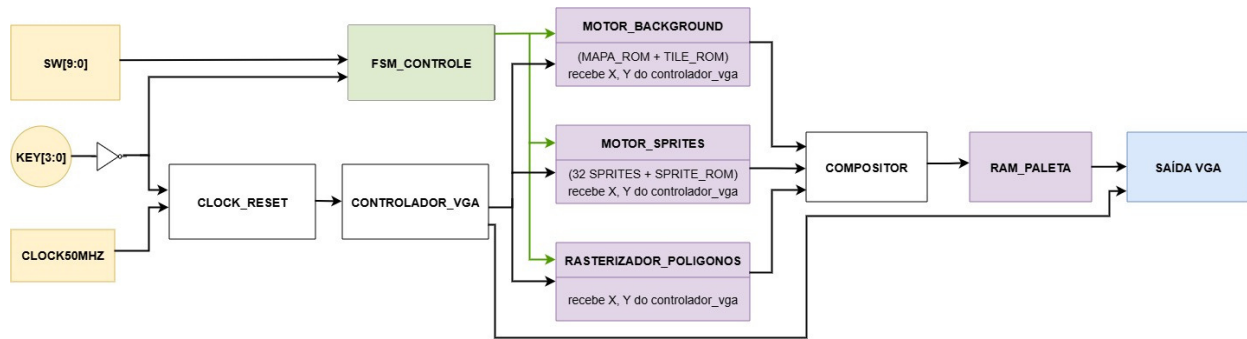


Figura 1 — Diagrama de blocos de alto nível do núcleo gráfico (main.v).

O fluxo de sinais do diagrama corresponde diretamente às instanciações em `main.v`:

- **CLOCK50MHZ, KEY[3:0] e SW[9:0]** são as entradas físicas da placa, mapeadas em `main.v` para os sinais internos `clock\_50mhz`, `key\_n` e `sw`.
- **clock_reset** (clock_reset.v) deriva o clock de vídeo de 25 MHz e sincroniza o reset (KEY0) para esse domínio.
- **controlador_vga** (controlador_vga.v) gera os pulsos de sincronismo HSYNC/VSYSNC, o sinal de área visível (blank) e as coordenadas físicas do pixel (0–639, 0–479), que são convertidas em `main.v` para as coordenadas lógicas 0–319/0–239.
- **fsm_controle** (fsm_controle.v) usa SW[9:8] para selecionar o modo ativo e isola, por modo, quais chaves/botões chegam a cada motor gráfico.
- **motor_background, motor_sprites e rasterizador_poligonos** recebem as mesmas coordenadas lógicas (X, Y) do `controlador\_vga` e calculam, em paralelo, o índice de cor de 8 bits que cada um produziria para aquele pixel.
- **compositor** (compositor.v) escolhe qual dos três índices de cor prevalece, por prioridade fixa.
- **ram_paleta** (ram_paleta.v) converte o índice de cor de 8 bits final em um valor RGB de 24 bits, enviado ao conector VGA (DAC ADV7123).

Um ponto importante, não evidente apenas pelo diagrama, é que o núcleo também contém uma interface de comandos de 32 bits (`gerador\_comandos\_demo.v` + `decodificador\_comandos.v`), instanciada em `main.v` e incluída na síntese, mas que hoje não está conectada aos motores gráficos — ver seção 12. Ela representa uma estrutura preparada para uso futuro, e não faz parte do caminho de dados que efetivamente gera a imagem exibida na demonstração atual.

4. Fluxo de Dados de um Pixel

Para cada pixel físico varrido pelo `controlador\_vga`, o sistema executa o seguinte fluxo, hoje inteiramente combinacional/pipelinado em poucos ciclos:

- **1. Varredura:** o `controlador\_vga` informa a coordenada física (X, Y) atual da varredura de 640x480.
- **2. Escalonamento:** `main.v` divide (X, Y) por 2 (`coord\_x\_vga[9:1]`, `coord\_y\_vga[9:1]`), obtendo a coordenada lógica de 320x240 usada por todos os motores.
- **3. Cálculo paralelo:** o `motor\_background` consulta suas duas ROMs para obter a cor de fundo; o `motor\_sprites` varre seus 32 registros de atributos em busca de colisão com o pixel; o `rasterizador\_poligonos` testa se o pixel está dentro do retângulo/triângulo capturado (via bounding-box ou funções de aresta).
- **4. Composição:** o `compositor` escolhe a cor final pela prioridade Polígono > Sprite > Background, tratando o índice 0 como transparente para as duas camadas superiores.
- **5. Decodificação de cor:** o índice de 8 bits resultante é expandido pela `ram\_paleta` para um valor RGB de 24 bits (8 bits por canal), no padrão RGB332, e enviado ao monitor caso o pixel esteja na área visível (`area\_visivel`/`VGA\_BLANK\_N`).

5. Máquina de Estados de Controle (fsm_controle.v)

A FSM de controle é puramente combinacional/registrada por chaveamento de modo (2 bits de estado) e não participa do datapath de vídeo em si — ela apenas decide quais sinais físicos (SW/KEY) são repassados a qual motor gráfico. O estado é definido diretamente por SW[9:8] (`modo\_selecionado`), sem lógica de transição condicionada a eventos:

SW9 SW8	Estado	Efeito
0 0	ESTADO_BACKGROUND (00)	SW[3:0] chegam ao motor_background; demais motores recebem chaves zeradas.
0 1	ESTADO_SPRITES (01)	KEY[3:1] e SW[7:0] chegam ao motor_sprites.
1 1	ESTADO_POLIGONOS (11)	KEY3 e SW[7:0] chegam ao rasterizador_poligonos.
1 0	(cai no default do case)	Não corresponde a nenhum dos três modos documentados nos slides; o case interno de fsm_controle trata esse valor pelo ramo default, que reproduz o comportamento do ESTADO_BACKGROUND (chaves de background zeradas, nenhum motor recebe controle específico).

Em qualquer modo, os motores não selecionados continuam sendo calculados normalmente a cada pixel (o `compositor` sempre recebe as três cores), porém deixam de receber novas entradas de chave — por isso o cenário, os sprites e os polígonos permanecem visíveis simultaneamente na tela mesmo quando apenas um deles está "ativo" para edição.

6. Motor de Background (motor_background.v)

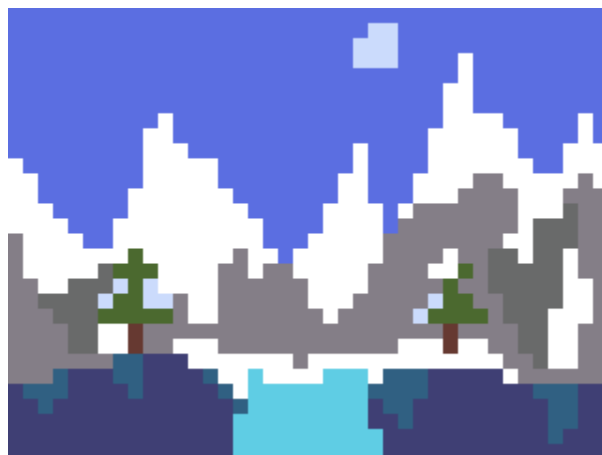


Figura 2 — Arte original do background (autoria: Bruna Nascimento), desenhada em resolução 320x240 antes da conversão para os índices de cor de 8 bits gravados em `tile_rom`.

O cenário de fundo é implementado como um tilemap lógico de 40x30 posições, cada uma ocupada por um tile de 8x8 pixels ($40 \times 8 = 320$, $30 \times 8 = 240$ — exatamente a resolução lógica do sistema). A hierarquia de memória usa duas ROMs, ambas sintetizadas com a IP "ROM: 1-PORT" (megafunção ``altsyncram``, bloco M10K):

- **mapa_rom** (`mapa_rom.v`, 1200 x 8 bits, inicializada a partir de `background_map.mif`): guarda, para cada uma das 1200 posições do mapa (40x30), o identificador (0–255) do tile que deve ser desenhado ali.
- **tile_rom** (`tile_rom.v`, 16384 x 8 bits, inicializada a partir de `background_tiles.mif`): guarda o desenho de 256 tiles possíveis, 64 pixels (8x8) cada — $256 \times 64 = 16384$ —, cada posição contendo o índice de cor RGB332 daquele pixel do tile.

Observação sobre a nomenclatura: o material de apresentação anterior referia-se a essas memórias como "tilemap_ram" e "tile_pattern_rom". No código atual, os módulos efetivamente instanciados em `motor_background.v` chamam-se ``mapa_rom`` e ``tile_rom``, e ambos são ROMs (não RAMs) com conteúdo fixo definido em tempo de síntese pelos arquivos `.mif` — não há escrita em tempo de execução nessas memórias na versão atual do projeto. Este relatório usa os nomes reais do código.

O endereço do mapa é calculado como ``tile_y * 40 + tile_x``, e o endereço do tile como ``{tile_id, pixel_y, pixel_x}`` (concatenação de $8+3+3 = 14$ bits). Como a leitura de ``mapa_rom`` introduz 1 ciclo de latência antes que o ``tile_id`` esteja disponível, o módulo atrasa ``pixel_x``/``pixel_y`` em um registrador (``pixel_x_d1``, ``pixel_y_d1``) para realinhar essas coordenadas com o ``tile_id`` correspondente antes de endereçar ``tile_rom`` — um pipeline de sincronia de 1 estágio.

A rolagem (scroll) do cenário é feita inteiramente em hardware: um contador de 20 bits gera um pulso periódico (a cada 400.000 ciclos de 25 MHz, $\approx 62,5$ Hz) que incrementa ou decrementa dois registradores

de deslocamento (``scroll_h``, ``scroll_v``) conforme as chaves SW3–SW0, com aritmética de wrap-around circular nas quatro bordas do mapa 320x240.

Arquivo não utilizado: o repositório também contém `tile_pattern_rom.v` (que referencia `background2.mif`, também 16384 x 8 bits). Ele não é instanciado em nenhum outro módulo e — mais decisivo — o arquivo `tile_pattern_rom.qip` não consta na lista de `QIP_FILE/VERILOG_FILE` do projeto Quartus (`PBL_1-SD_grupo_4.qsf`); apenas `sprite_rom.qip`, `tile_rom.qip` e `mapa_rom.qip` são referenciados. Esse módulo e o respectivo `.mif` são, portanto, artefatos remanescentes que não fazem parte do design atualmente sintetizado.

7. Operação: Estado A — Background

Ativação: SW9 = 0, SW8 = 0.

Entrada	Função
SW3	Rola o cenário para a esquerda
SW2	Rola o cenário para baixo
SW1	Rola o cenário para cima
SW0	Rola o cenário para a direita
KEY0	Reseta o sistema (reset físico, ativo em nível baixo)

8. Motor de Sprites (`motor_sprites.v`)

O motor controla até 32 entidades simultâneas de 16x16 pixels. Diferentemente do background, os atributos de cada sprite não residem em uma memória RAM dedicada: são vetores de registradores internos (``sprite_x``, ``sprite_y``, ``sprite_pat``, ``sprite_en``, ``sprite_mirrored_h``, ``sprite_mirrored_v``, ``sprite_transp``, cada um com 32 posições), o que permite ler os 32 atributos em um único ciclo de clock durante a varredura de prioridade.

A imagem (padrão de pixels) de cada um dos 32 sprites vem de uma única ROM dedicada, ``sprite_rom`` (`sprite_rom.v`, 8192 x 8 bits, inicializada por `sprite_patterns.mif`): 32 padrões x 256 pixels (16x16) = 8192. O endereço de leitura é `{pat_local, py_local, px_local}` — o índice do padrão concatenado com a posição do pixel dentro do sprite de 16x16.

Regra de prioridade e busca por colisão

A cada pixel, um laço combinacional percorre os índices de 0 a 31; o primeiro sprite habilitado (``sprite_en``) cuja área de 16x16 contenha a coordenada (X, Y) atual é o que "vence" — ou seja, o sprite de menor índice tem prioridade máxima e fica por cima dos demais. Não há teste de profundidade ou reordenação: a prioridade é fixa pela ordem do vetor.

Controles no modo Sprites

- **SW[4:0] — seleção do sprite (0 a 31):** define qual entrada dos vetores de atributos recebe as próximas edições e movimentos.
- SW7 — transparência do sprite selecionado.
- SW6 / SW5 — espelhamento horizontal / vertical do sprite selecionado.
- KEY1 — alterna qual par de direções KEY3/KEY2 controla: no modo inicial, KEY3 move para a esquerda e KEY2 para baixo; após pressionar KEY1, KEY3 passa a mover para a direita e KEY2 para cima.
- KEY3 / KEY2 — movimentam o sprite selecionado a ~70 Hz (pulso gerado por um contador de 19 bits), com a posição limitada para manter o sprite dentro da área de 320x240.
- KEY0 — reset do sistema.

Correção sobre a cor dos sprites

Uma versão anterior do material do projeto apontava, como problema, uma suposta mudança de cor dos sprites ao serem selecionados. A leitura do código atual não confirma esse comportamento como uma limitação: a seleção via SW[4:0] apenas determina qual entrada dos vetores de atributos recebe as chaves de edição (SW7/SW6/SW5) e os botões de movimento (KEY3/KEY2). A cor de cada pixel do sprite é sempre obtida diretamente de ``sprite_rom``, endereçada pelo padrão (``sprite_pat``), sem qualquer dependência do índice de seleção atual — ou seja, o desenho e as cores do sprite não são alterados pelo simples ato de selecioná-lo. Por esse motivo, esta versão do relatório não reproduz aquela descrição como um problema do projeto.

O que de fato existe, e que pode ser confundido com "mudança de cor", é o mecanismo de transparência: o índice de cor 0 dentro de ``sprite_rom`` é sempre tratado como transparente (deixa a camada de baixo aparecer), independentemente do estado de SW7 — a lógica ``(!pixel_encontrado || transp_local || (cor_rom == 8'd0))`` força o índice de saída para 0 nesses três casos. Já a chave SW7 (armazenada em ``sprite_transp``) controla, adicionalmente, a transparência do sprite inteiro: quando ativada para o sprite selecionado, todo o sprite fica invisível, independentemente do conteúdo de ``sprite_rom`` — um recurso pensado para a demonstração, e não uma falha de renderização.

9. Operação: Estado B — Sprites

Ativação: SW9 = 0, SW8 = 1.

Entrada	Função
SW[4:0]	Seleciona o sprite a editar (0 a 31)
SW7	Transparência do sprite selecionado

SW6 / SW5	Espelhamento horizontal / vertical do sprite selecionado
KEY1	Alterna o par de direções controlado por KEY3/KEY2
KEY3 / KEY2	Move o sprite selecionado (esquerda/direita ou cima/baixo, conforme KEY1)
KEY0	Reseta o sistema

10. Rasterizador de Polígonos (rasterizador_poligonos.v)

Este motor permite desenhar, em tempo real, um retângulo ou um triângulo preenchido, definidos por 2 ou 3 vértices capturados interativamente. Um cursor lógico independente — não vinculado a nenhum sprite — percorre a tela a partir da posição inicial (160, 120), a uma taxa fixa de ~100 atualizações por segundo (contador de 19 bits), podendo mover-se na diagonal se duas chaves de eixos diferentes estiverem ativas ao mesmo tempo.

O botão KEY3 (com sincronização de dois flip-flops, detecção de borda e bloqueio anti-bouncing de ~10 ms) marca a posição atual do cursor como um vértice. Uma pequena máquina de estados de captura (`estado\_captura`: 0→1→2) acumula os vértices: se SW7 = 1 (retângulo), o polígono fica pronto após o segundo vértice (P1, P2, opostos do retângulo); se SW7 = 0 (triângulo), são necessários três vértices (P1, P2, P3). O tipo é travado (`tipo\_latched`) no instante em que o primeiro vértice é marcado, evitando que uma mudança de SW7 no meio da captura corrompa a forma em construção.

O teste de preenchimento difere por tipo de forma:

- **Retângulo:** teste de bounding-box simples com min/max de X e Y entre os dois vértices.
- **Triângulo:** teste por funções de aresta (edge functions) com aritmética inteira com sinal — $E(A,B,P) = (Px-Ax)(By-Ay) - (Py-Ay)(Bx-Ax)$ para cada uma das três arestas. O pixel é preenchido quando as três funções têm o mesmo sinal (todas ≥ 0 ou todas ≤ 0), o que cobre corretamente os dois sentidos de definição dos vértices (horário ou anti-horário).

O próprio cursor é desenhado com prioridade máxima, como um único pixel lógico de índice de cor 255 (branco, conforme detalhado na Tabela 2). As cores de preenchimento são fixas em código: índice 100 para retângulos e índice 224 para triângulos.

Elemento	Índice (8 bits)	Cor resultante após RGB332 (ram_paleta)
Cursor	255 (0xFF)	Branco — RGB(255, 255, 255)
Preenchimento do retângulo	100 (0x64)	Laranja escuro/marrom — RGB(109, 36, 0)
Preenchimento do triângulo	224 (0xE0)	Vermelho puro — RGB(255, 0, 0)

Tabela 2 — Cores fixas geradas pelo rasterizador de polígonos, calculadas a partir da lógica de expansão de bits de ram_paleta.v.

11. Operação: Estado C — Polígonos

Ativação: SW9 = 1, SW8 = 1.

Entrada	Função
SW7	Tipo de polígono: 0 = triângulo, 1 = retângulo
SW[3:0]	Move o cursor (esquerda, baixo, cima, direita)
KEY3	Marca a posição atual do cursor como vértice
KEY0	Reseta o sistema

12. Compositor e Conversão de Cores

Compositor (`compositor.v`)

O compositor é um bloco puramente combinacional que aplica uma prioridade fixa entre as três camadas: Polígono > Sprite > Background. Os índices de cor 0 vindos do sprite e do polígono são tratados como transparentes; a camada de background nunca é considerada transparente, funcionando como o plano de fundo padrão sempre visível.

Conversão de índice para RGB (`ram_paleta.v`)

Apesar do nome herdado "ram_paleta", o módulo atual não contém nenhuma memória: não há arquivo de inicialização, nem uma tabela de cores arbitrária armazenada. Trata-se de um decodificador síncrono que fatia diretamente o índice de 8 bits em três campos — 3 bits de vermelho, 3 bits de verde e 2 bits de azul (padrão RGB332) — e expande cada campo para 8 bits por replicação dos próprios bits (bit-replication), registrando o resultado de 24 bits a cada ciclo de clock de pixel. Ou seja, qualquer índice de 8 bits produzido pelas camadas gráficas é mapeado, de forma fixa e determinística, para uma entre 256 cores possíveis desse espaço de cores — não há indireção de paleta nem possibilidade de remapear cores em tempo de execução na versão atual.

Essa característica é a origem direta da limitação de cores do background, tratada na seção 17.

13. Geração de Clock e Sincronismo de Vídeo

`clock_reset.v`

Divide o clock de 50 MHz da placa por 2 através de um flip-flop de alternância (toggle), gerando o clock de vídeo de 25 MHz. O reset físico (KEY0, ativo em nível baixo) é sincronizado para esse domínio de clock por uma fila de dois flip-flops, prática padrão para evitar metaestabilidade.

`controlador_vga.v`

Implementa a temporização padrão de VGA 640x480 a partir de contadores horizontal e vertical, com os seguintes parâmetros (em pixels/linhas, a 25 MHz):

Parâmetro	Horizontal (pixels)	Vertical (linhas)
Área ativa	640	480
Front porch	16	10
Pulso de sincronismo	96	2
Back porch	48	33
Total	800	525

Com $800 \times 525 = 420.000$ ciclos de clock por quadro a 25 MHz, a taxa de atualização resultante é de aproximadamente 59,5 Hz, compatível com o padrão VGA 640x480 @ 60 Hz. Os sinais HSYNC, VSYNC e a área visível (blank) são registrados (e não combinacionais) para evitar glitches na saída; as coordenadas de pixel exportadas (`o\_x`, `o\_y`) são limitadas para nunca ultrapassar a área ativa.

14. Escalonamento de Resolução (640x480 → 320x240)

A resolução lógica interna usada por todos os motores gráficos é 320x240 — metade da resolução física do VGA em cada eixo. A conversão é feita em `main.v` descartando o bit menos significativo das coordenadas físicas: `coord\_logica\_x = coord\_x\_vga[9:1]` e `coord\_logica\_y = coord\_y\_vga[9:1]`, equivalente a uma divisão inteira por 2 sem custo de hardware. Como consequência, cada pixel lógico é fisicamente exibido como um bloco de 2x2 pixels na tela, dobrando o tamanho aparente de toda a arte (tiles, sprites e polígonos) sem necessidade de replicar os dados armazenados nas ROMs.

15. Depuração: LEDs e Displays de 7 Segmentos

Além da saída de vídeo, `main.v` expõe informações de depuração nos periféricos de status da placa:

- **LEDR[9:8]** exibem o modo atual selecionado por SW[9:8]; LEDR[7:0] espelham diretamente SW[7:0].
- **HEX0–HEX2** exibem, em hexadecimal, a coordenada X de depuração; HEX3–HEX4 exibem a coordenada Y; HEX5 exibe o estado atual da FSM.

O conteúdo exato de X/Y depende do modo ativo: no estado de sprites, mostram a posição do sprite selecionado (`sprite\_x\_debug`/`sprite\_y\_debug`); no estado de polígonos, mostram a posição do cursor (`cursor\_x\_debug`/`cursor\_y\_debug`); no estado de background (ou no valor de seletor não mapeado, SW9=1/SW8=0), mostram o próprio valor das chaves SW[3:0] de controle do scroll. A conversão para os segmentos físicos é feita pelo módulo genérico `decodificador\_hex.v`, instanciado seis vezes.

16. Memórias do Sistema

A tabela a seguir resume todas as memórias realmente instanciadas e sintetizadas no projeto atual (conforme confirmado em `PBL\_1-SD\_grupo\_4.qsf`), além dos vetores de atributos dos sprites, que não são memórias de bloco, mas cumprem papel equivalente.

Memória	Tipo	Tamanho	Conteúdo	Módulo(s) que usa(m)	Acesso
mapa_rom	ROM	1200 x 8 bits	ID do tile (0–255) em cada uma das 1200 posições do mapa 40x30 (background_map.mif)	motor_background	Leitura síncrona, 1 porta
tile_rom	ROM	16384 x 8 bits	Índices de cor de 256 tiles de 8x8 pixels (background_tiles.mif)	motor_background	Leitura síncrona, 1 porta
sprite_rom	ROM	8192 x 8 bits	Índices de cor de 32 padrões de sprite de 16x16 pixels (sprite_patterns.mif)	motor_sprites	Leitura síncrona, 1 porta
Vetores de atributos de sprite	Registradores (não RAM de bloco)	32 posições por vetor (x, y, padrão, ativo, espelho H/V, transparência)	Estado dinâmico de cada um dos 32 sprites, atualizado em tempo real	motor_sprites	Leitura/escrita a cada ciclo
ram_paleta	Não é memória	—	Nenhum; decodifica o índice RGB332 por lógica combinacional/registrada	main (entre compositor e VGA)	Combinacional, registrada por ciclo
tile_pattern_rom (+ background2.mif)	ROM (não sintetizada)	16384 x 8 bits	Padrões alternativos de tile, não referenciados pelo projeto Quartus atual	Nenhum módulo instancia	Não aplicável — arquivo não incluído no .qsf

17. Interface de Comandos de 32 bits (estrutura preparada para uso futuro)

O projeto inclui, na síntese, uma interface de comandos de 32 bits pensada para permitir que um processador externo (por exemplo, um núcleo ARM/HPS) controle o coprocessador gráfico por software:

- **gerador_comandos_demo.v** simula esse processador: a cada ~0,5 segundo (contador de 24 bits a 25 MHz), emite um de cinco comandos de demonstração pré-definidos, alternando ciclicamente pelos opcodes OP_CONFIG_BG, OP_SET_TILE, OP_CONFIG_SPR, OP_RASTER_POLY e OP_WRITE_PAL.
- **decodificador_comandos.v** recebe o comando de 32 bits e o strobe `valido`, e apenas fatia e registra internamente o opcode e os parâmetros A/B/C. O módulo não possui nenhuma porta de saída.

Efeito prático: como `decodificador_comandos.v` não expõe saídas, seus registradores internos não chegam a nenhum outro módulo do projeto — em particular, não há conexão com `motor_background`, `motor_sprites`, `rasterizador_poligonos` nem `ram_paleta`. Isso significa que, na versão atual, os comandos gerados por `gerador_comandos_demo` não produzem nenhum efeito visível na imagem: toda a cena exibida na demonstração é controlada exclusivamente pelas chaves e botões físicos, por meio da `fsm_controle`. Essa interface de comandos deve ser entendida como uma estrutura de hardware já implementada e sintetizada, preparada para uma futura integração com um processador real, mas ainda não conectada ao restante do datapath gráfico.

18. Módulos Presentes no Repositório mas Não Utilizados

A partir da conferência cruzada entre o código-fonte, o arquivo de projeto Quartus (``PBL_1-SD_grupo_4.qsf``) e os arquivos `.qip``, foram identificados os seguintes itens que existem no repositório mas não fazem parte do design efetivamente sintetizado ou não afetam a imagem exibida:

- **tile_pattern_rom.v e background2.mif** — ROM alternativa para os padrões de tile (16384 x 8 bits). O arquivo `tile_pattern_rom.qip` não é referenciado em `PBL_1-SD_grupo_4.qsf` (apenas `sprite_rom.qip`, `tile_rom.qip` e `mapa_rom.qip` são), e nenhum módulo Verilog do projeto instancia `tile_pattern_rom`. É um artefato remanescente, não sintetizado.
- **mapa_rom_bb.v, tile_rom_bb.v e sprite_rom_bb.v** — declarações "black-box" geradas automaticamente pelo Quartus, listadas como `MISC_FILE` nos respectivos `.qip`. Servem apenas como metadado/estímulo para simulação e visualização de IP; a lógica real está nos módulos `altsyncram` dos arquivos `.v` principais (`mapa_rom.v`, `tile_rom.v`, `sprite_rom.v`), que são os efetivamente sintetizados.
- **decodificador_comandos.v e gerador_comandos_demo.v** — são sintetizados e fazem parte do projeto (constam no `.qsf`), mas, como detalhado na seção 17, estão estruturalmente desconectados do caminho de renderização — presentes como base para um trabalho futuro, sem efeito sobre a imagem gerada hoje.

Não confirmado pelos arquivos fornecidos: o `.qsf` referencia dois testbenches de simulação (`testbenches/tb_rasterizer.v` e `tb_background_engine.v`, seção `EDA_TEST_BENCH_FILE`), mas os arquivos desses testbenches não estavam entre os arquivos disponibilizados para este relatório. Portanto, seu conteúdo e cobertura de testes não são descritos aqui.

19. Pinagem Geral (DE1-SoC)

Sinal	Pino físico	Função
CLOCK_50	PIN_AF14	Clock de 50 MHz da placa
KEY[0]	PIN_AA14	Reset físico do sistema (ativo em nível baixo)

KEY[1]	PIN_AA15	Depende do modo (ex.: alterna direções no modo Sprites)
KEY[2]/KEY[3]	PIN_W15 / PIN_Y16	Dependem do modo (movimento de sprite ou marcação de vértice)
SW[9:8]	— (ver .qsf)	Seleção do modo de operação da FSM
SW[7:0]	— (ver .qsf)	Função dependente do modo ativo (scroll, atributos de sprite ou controle de polígono/cursor)
LEDR[9:0]	— (ver .qsf)	Depuração: modo atual e espelho de SW[7:0]
HEX0–HEX5	— (ver .qsf)	Depuração: coordenadas X/Y e estado da FSM, em hexadecimal
VGA_R/G/B, VGA_HS/VS, VGA_CLK, VGA_BLANK_N, VGA_SYNC_N	— (ver .qsf)	Saída de vídeo para o DAC VGA (ADV7123) da placa

Os pinos de SW, LEDR e HEX individuais seguem o padrão da placa DE1-SoC e estão listados integralmente em PBL_1-SD_grupo_4.qsf; a tabela acima resume apenas os sinais de mais alto nível para não repetir as 40+ atribuições individuais do arquivo de restrições. Vale reforçar que o topo físico `DE1\_SOC\_golden\_top.v` — modelo padrão da Terasic para a placa — declara diversos outros periféricos (ADC, áudio, DRAM, GPIO, HPS, IRDA, PS2, TD) que não são conectados ao módulo `main` do projeto: apenas CLOCK_50, KEY, SW, LEDR, HEX0–5 e o barramento VGA são efetivamente usados pelo coprocessador gráfico.

20. Arquitetura Atual: o que Está Implementado, em Uso, ou Preparado

Categoria	Itens
Implementado e usado na demonstração	clock_reset, controlador_vga, fsm_controle, motor_background (com mapa_rom e tile_rom), motor_sprites (com sprite_rom), rasterizador_poligonos, compositor, ram_paleta, decodificador_hex — todo o caminho de chaves/botões até a saída VGA.
Implementado, sintetizado, mas não exercido na demonstração atual	gerador_comandos_demo e decodificador_comandos (interface de comandos de 32 bits, sem conexão às camadas gráficas — ver seção 17).
Presente no repositório, não sintetizado	tile_pattern_rom.v e background2.mif (ROM alternativa de tiles, fora do .qsf).
Não faz mais parte do projeto como problema	A suposta alteração de cor dos sprites ao serem selecionados, mencionada em material anterior — não confirmada pelo código atual (ver seção 8).

21. Limitações e Trabalhos Futuros

Limitação real: espaço de cores RGB332 do background

A única limitação de cor confirmada no projeto atual está relacionada à representação de cores da saída de vídeo, e não a um defeito do motor gráfico, do compositor ou dos sprites. O background (Figura 2) foi desenhado manualmente com cores livres de 24 bits; para ser exibido pelo sistema, cada pixel do tile precisou ser convertido para um único índice de 8 bits, decodificado por ``ram_paleta`` no padrão RGB332 (3 bits de vermelho, 3 de verde, 2 de azul — apenas 256 combinações possíveis, contra os mais de 16 milhões de cores do desenho original). Como consequência, algumas cores específicas escolhidas originalmente para o cenário não têm correspondência exata nesse espaço de 256 cores e precisam ser aproximadas para o valor RGB332 mais próximo disponível — o que gera diferenças visuais perceptíveis entre a arte de referência e o resultado exibido no monitor. Esta é uma limitação de representação de cores do sistema, não um erro de funcionamento do motor de background nem dos sprites.

Ausência de troca de paleta em sprites e polígonos

Sprites e polígonos utilizam índices de cor fixos e diretos (ex.: 100 e 224 no rasterizador), sem qualquer mecanismo de remapeamento de paleta por objeto. Um trabalho futuro natural seria permitir que cada sprite ou polígono apontasse para uma paleta própria, ampliando a variedade de cores sem aumentar a largura de cada índice.

Interface de comandos ainda não conectada

A interface de comandos de 32 bits (seção 17) já está implementada e sintetizada, mas `decodificador_comandos.v` não possui saídas conectadas aos motores gráficos. Conectar essas saídas aos registradores de scroll do background, aos atributos dos sprites e aos vértices do rasterizador é o passo natural para permitir que um processador externo (HPS/ARM) controle a cena por software, em vez de depender apenas de chaves físicas.

22. Conclusão

O núcleo de coprocessador gráfico atualmente implementado na DE1-SoC cumpre integralmente a proposta de gerar, de forma autônoma, um sinal de vídeo VGA de 640x480 @ ~60 Hz a partir de um datapath de 320x240 com três camadas gráficas independentes — background com rolagem por hardware, até 32 sprites com espelhamento e transparência configuráveis, e um rasterizador de retângulos e triângulos com captura interativa de vértices —, todas compostas por um esquema de prioridade fixo e convertidas para RGB por um decodificador RGB332. O controle de todo o sistema é feito diretamente pelas chaves e botões da placa, isolados por modo através de uma pequena máquina de estados.

A revisão do código-fonte confirmou que a arquitetura de memórias (três ROMs para tiles, mapa e sprites), a lógica de prioridade dos sprites e o comportamento do compositor correspondem ao que é descrito neste relatório, e permitiu corrigir duas imprecisões do material anterior: (i) não existe alteração

de cor dos sprites ao serem selecionados — o desenho de cada sprite é sempre lido diretamente de `sprite_rom`, independentemente da seleção; e (ii) a limitação de cor real e observável do projeto está no espaço de cores RGB332 usado para converter a arte do background, e não em um defeito do motor gráfico. A principal limitação pendente é a falta de conexão entre a interface de comandos de 32 bits — já implementada — e os motores gráficos, o que hoje impede que um processador externo controle a cena por software; esse é o passo mais natural para a continuidade do projeto.

Referências

DAUM, Michael. Terasic DE1-SoC Development and Education Board. RocketBoards, 2016. Disponível em: <https://www.rocketboards.org/foswiki/Documentation/TerasicDE1SoCDevelopmentAndEducationBoard>. Acesso em: 08 set. 2026.

TERASIC. DE1-SoC User Manual, rev. F. Terasic Technologies Inc., [S.l.], 2018.